

Abstract

Text generation has progressed from earlier statistical models to advanced neural networks. Earlier approaches were limited by data sparsity, weak semantic understanding, and subpar baseline performance, including n-gram and HMM models. Newer models, such as RNNS and LSTM, have improved sequence modelling by incorporating hidden states and addressing vanishing gradients. This shift emerged with Transformer architecture, introduced in “Attention Is All You Need” (2017), which made the basis of transformers, self-attention, and parallelization. Transformers have acted as the backbone of models like GPT and BERT for bidirectional understanding. These models have been used in conversational agents like ChatGPT, Gemini, and GitHub Copilot for code generation. Despite this success, it has serious drawbacks, including scalability and ethical deployment. Studies are in progress to improve transformers as they will remain the centre of future progress in NLP.

Introduction

Text generation is a process of using Artificial Intelligence (AI) to generate human-like content. The generation harnesses machine learning techniques involving Natural Language Processing (NLP), Hidden Markov Models (HMMs), and

advanced generative techniques, including Variational Autoencoders (VAEs) and Reinforcement learning. (Bengio et al., 2003; Hochreiter & Schmidhuber, 1997).

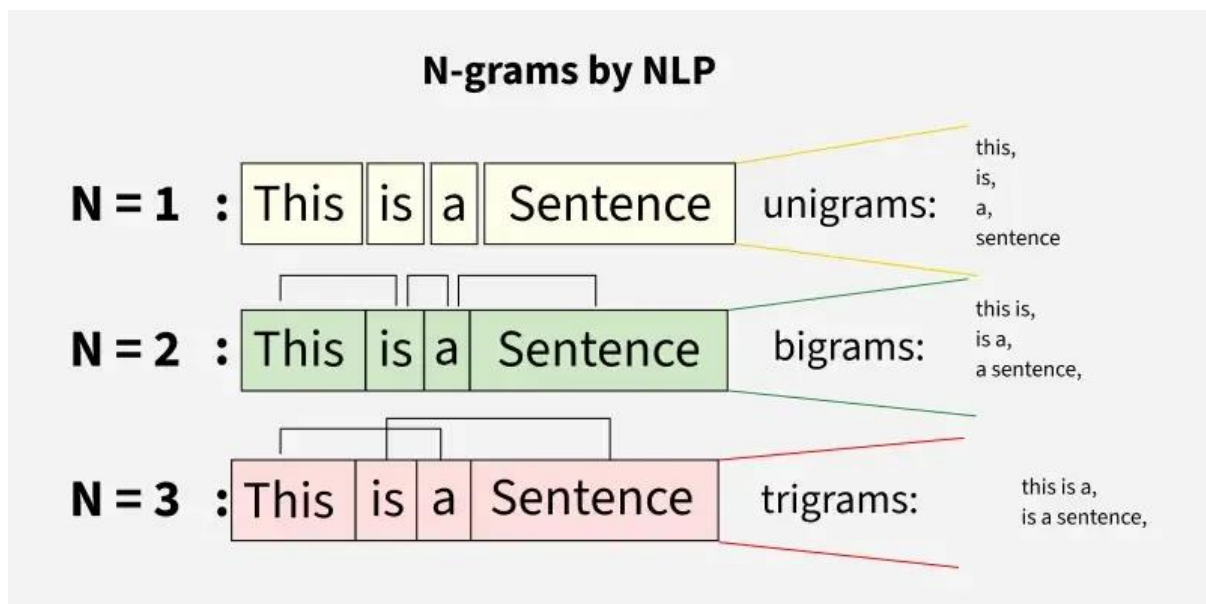
Early methods and Limitations

The early methods of text generation included numerous techniques, including n-gram and HMMs. An n-gram is a technique that predicts the word that would likely be used, determined from the structure of sentences. An n-gram looks at the sequences of the words of length n that appear together in a corpus. There are mainly three types of n-gram models: Unigram, Bigram, and Trigram. Unigram takes input words and outputs one predicted word. We can say that it uses the formula $p(w|h)$ where w is the next word and h is the context history.

Example: For the phrase “This research paper is on” The model will predict the likelihood of the next word like:

$p(\text{"Transformers"} | \text{"This", "research", "paper", "is", "on"})$

The bigram model uses a pair of consecutive words, and the trigram model uses a three-word sequence, offering more context and detailed meaning.



Sourced by [Geeksforgeeks](#).

The benefits of n-gram were considerable. N-gram requires minimal computational power with increased simplicity. This helps to provide a strong

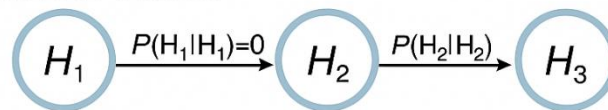
baseline for foundation modelling and quick prototyping. Moreover, n-gram excels in contextual understanding. Language is relatively complex; it changes its meaning according to the given context. N-gram distinguishes itself by preserving the deeper meaning of text that single-word analysis would overlook. However, we cannot ignore the limitations n-gram brings with itself. Data sparsity is one of the problems. In simple words, data sparsity is a state where a large amount of data is missing in a dataset or is set to zero. N-gram finds it difficult to find repeated instances of the sequence, which leads to sparse data. Furthermore, n-gram also faces issues related to semantic understanding. Semantic understanding refers to the ability to comprehend the meaning and context of the communication. While n-gram models are good at recognizing patterns,

they lack the understanding of the context they are not trained on. Lastly, n-gram only considers nearby words to predict the words. Like in the example stated, an n-gram only considers nearby elements of the sentence. Thus, neglecting the broader meaning of the text.

Moving ahead, another model that is used in text generation is the Hidden Markov model (HMMs). This model runs on a Markov model in which evaluation is dependent on a Markov process (referred to as X). HMM requires that there should be an observable process Y whose outcomes are dependent on X . Since X is not observable, the goal becomes to learn about the state of X by scrutinizing Y .

HIDDEN MARKOV MODEL (HMM): Connecting the unseen with the observed

HIDDEN STATES



We CANNOT see these states directly. We only know probabilities of transitioning between them.

OBSERVED STATES



THE CHALLENGE:

Given a sequence of observations (e.g. sun, music, rain), infer the most likely sequence of hidden states that produced them

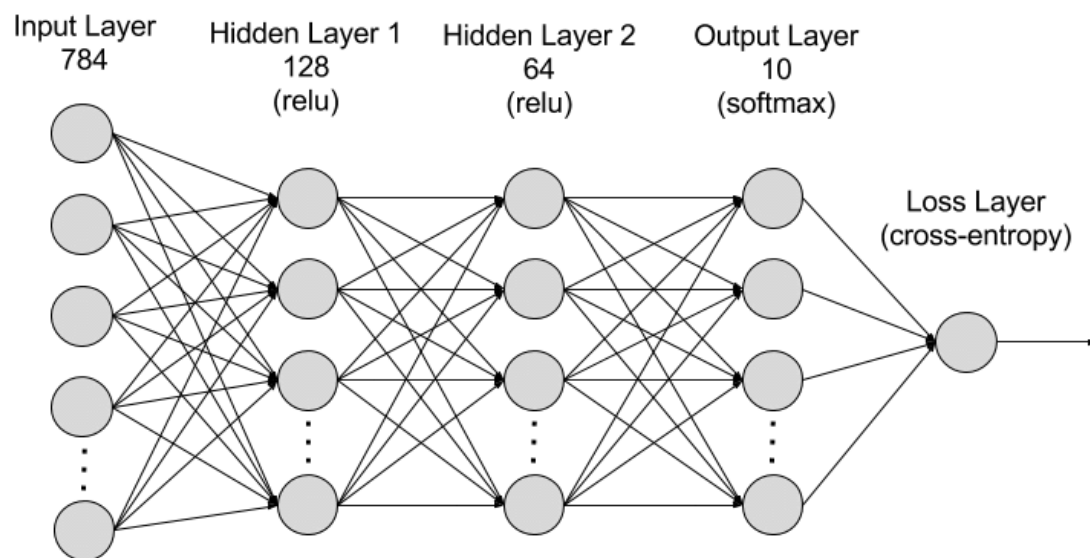
Undoubtedly, HMMs have numerous benefits, including capturing sequential structure. HMMs naturally model the orders of words. By using grammatical probabilities, these models can generate text that follows grammatical-like patterns instead of random word placement. Moreover, HMMs have strong contextual understanding. HMMs incorporate some context by conditioning each word on the hidden state and the previous state, even though it is limited. This allows for a more coherent text in comparison to other models like n-grams. Furthermore, every time you use HMM, it

will generate a different set of sentences due to its probabilistic nature. This results in more diversity than deterministic grammar-based systems. HMMs also come with drawbacks, including limited memory because of the Markov Assumption. In this assumption, the model predicts the next state from the current stage, not the entire history. This means that HMMs cannot be used in long-term dependencies in text. For Example, in the sentence *"The boy who was playing in the park is very happy,"* the word *"is"* must agree with *"boy,"* not with the immediately preceding word *"park."*

HMMs often fail to capture such long-range grammatical dependencies. In natural language, there is a need to store a large number of words, ranging from tens of thousands to millions of words. HMMs represent language using transition probabilities between hidden states and emission probabilities of words given states. This means that when the vocabulary grows, these matrices need more training data to estimate probability reliably. This results in data sparsity, which means that HMMs are not reliable for use on unseen data. Moreover, HMMs oversimplify the independent assumption. This means that it cannot understand complex sentences. Like, the meaning of “bank” in “He sat by the bank” depends on the context that

spans multiple words, something an HMM cannot capture effectively.

Besides this, another text generation technique includes recurrent neural networks (RNNs). RNN works by remembering past information, which helps the network understand and make better predictions. These models have three components: input, output, and hidden layer. The input layer receives the data, and the output layer offers the result. The hidden layer does the processing required, analysis, and prediction of data. In addition to that, there are 4 types of RNNs: One-to-One, One-to-Many, Many-to-One, and Many-to-Many RNNs.



From AWS (Amazon Web Services)

The benefits of RNN cannot be ignored. RNN provides sequential data handling. It means that RNN processes the data one step at a time, which means that it is ideal for tasks that require order, including text generation. Furthermore, as mentioned earlier, RNNs can retain the past information that influences their predictions. RNNs have a hidden state that carries information about previous words in a sequence, which allows them to produce a better prediction than other models mentioned above. Other than that, RNN uses the same weights at every time step, reducing the number of parameters and improving generalisation across positions in the sequence. However, the drawbacks of RNN are not less. RNNs have a problem related to vanishing and exploding gradients, in which RNNs use backpropagation

through time (BPTT), leading to vanishing gradients. Making it hard to learn long-range dependencies. RNNs also suffer from short-term memory, as standard RNNs are not good at long-term memory performance. This limits their ability to maintain context and coherence across long passages. Moreover, RNNs require a large amount of time to learn as they learn step by step. RNNs also have a problem capturing complex patterns because it is a comparatively simple form compared to the rest of the models. RNNs are harder to tune and optimize. Compared to transformers, RNNs often require more careful tuning of hyperparameters. This can sometimes lead to inferior performance in comparison to other models.

It also has another model known as Long Short-Term Memory (LSTM). This was introduced to

mitigate the problem related to the vanishing gradient problem in RNNs. It almost acts the same as RNNs but uses input, output, and forget gates instead. LSTMs cope with the discrepancies in RNNs like long-term dependencies, gradient stability, and complexity. But it also has problems. LSTMs suffer from computational complexity. As LSTMs use gates, a large number of parameters are needed to carry out calculations. This results in slower training and inference compared to vanilla RNNs. LSTMs also do not allow parallel processing. It is inherently sequential, so each time step depends on the previous one, making the training slower even with more GPUs. Furthermore, over-fitting is another problem. Due to high capacity, LSTMs can overfit on small datasets. Human intelligence is another considerable problem. As the internal working of LSTMs is carried out by gates, it makes the process hard to interpret, making it harder to understand what the model is “remembering” or “forgetting.”

The Rise of Transformers

Transformers are a deep learning architecture introduced in the paper “Attention is All You Need” (2017) by Vaswani et al. They are the foundation of

modern natural language processing (NLP) and are also widely used in vision, speech, and multimodal AI systems. The goal of transformers was to reduce sequential computation using ByteNet and ConvS2S. These use convolutional neural networks as basic building blocks, processing the inputs and outputs at the same time. In previous models, the number of operations required to relate two arbitrary inputs and outputs to learn, linearly for ConvS2S and logarithmically for ByteNet. Transformers act on self-attention, something called out to be intra-attention. Self-attention allows each element (token) in a sequence to attend to or interact with all other elements of the same sequence to compute a representation of that element, capturing the relationship regardless of their position. For example, in the sentence “Cat sat on the mat”. Transformers can capture the relationship between cat and mat even though there are three additional words between them. In addition, transformers use end-to-end memory networks that are based on a recurrent attention mechanism instead of sequence-aligned recurrence, which performs better at simple-language question answering and language modelling tasks.

Working Principle

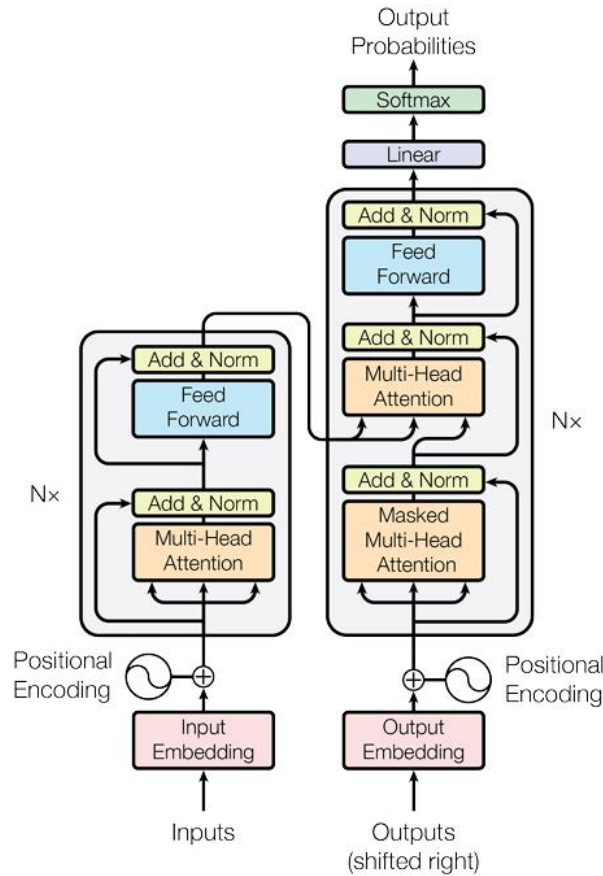


Figure 1: The Transformer - model architecture.

From "Attention Is All You Need"

Transformers work on some basic concepts. One of which is the Self-Attention Mechanism. This mechanism allows it to determine which words in a sentence are relevant and which are not. This is executed using the dot-product attention approach. Each of the words is mapped to three vectors: Query (Q), Key (K), and Value (V). This is computed as: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$. Moreover, each word is converted into a high-dimensional vector using an embedding layer. Since transformers use parallel processing, they lack the understanding of the order of data. To counter this, positional encoding is used. This uses sine and cosine functions because they provide a smooth encoding of position, allowing easy interpolation and generalization across sequences for varying lengths. The following formulas are used in this calculation:

1. Even-indexed dimensions:

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{(2i/d_{model})}}\right)$$

1. Odd-indexed dimensions:

$$PE_{pos,2i} = \cos\left(\frac{pos}{10000^{(2i/d_{model})}}\right)$$

Where:

- $PE_{pos,2i}$: The positional encoding at position pos for the $2i$ th dimension.
- Pos : The position of the token in the sequence (starting from 0).
- i : The index of the dimension within the positional encoding vector (for each i , we have two formulas: one for $2i$ (sine) and another for $2i+1$ (cosine)).
- d_{model} : The dimensionality of the model (the embedding size, e.g., 512, 1024, etc).
- The exponential term $10000^{(2i/d_{model})}$: This controls how the sine wave's frequency changes with the dimension.

Besides this, transformers also use Multi-Head Attention. In this process, the model has different heads running in parallel. Each head captures a

different relationship and patterns in data, enhancing the understanding of the model. The transformer model contains a Position-wise-Feed-Forward Network (FFN) in every layer (both encoder and decoder). It helps the model to understand context by exchanging information, which adds non-linearity and helps the model to transform that context into a higher-level representation. It is like attention is for “gathering context” and FFN is for “processing the gathered context.” It consists of two fully connected (dense) layers with activation energy in between. The formula for this process is:

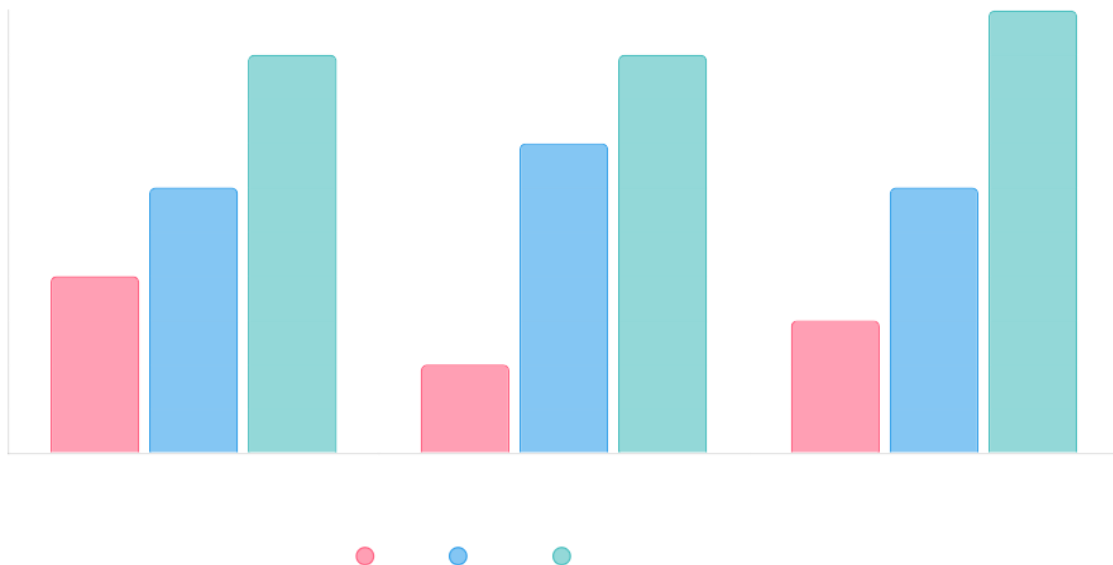
$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

Where:

- x is the input token representation (a vector)
- W_1, W_2 , and b_1, b_2 are trainable parameters (weights and biases)

- **ReLU** is a non-linear activation function (sometimes replaced with GELU in newer models)

Encoder-Decoder Architecture is also used in transformers. It mainly has a two-part neural network design. The encoder processes the data and takes out key valuable insights. The decoder is used for the output sequence. The encoder receives the token input (likely in English) and outputs a contextual representation of each token, considering the whole sequence. The decoder generates the output, one token at a time. The decode models include: GPT, GPT-2, GPT-3, GPT-4, PT-Neo / GPT-J / GPT-NeoX, LLaMA / LLaMA 2 / LLaMA 3, Falcon, and more. Furthermore, the encoder models include: BERT, RoBERTa, ALBERT, DistilBERT, ELECTRA, DeBERTa / DeBERTaV2, CamemBERT, XLM, XLM-RoBERTa, MPNet, Funnel Transformer, and more.



Transformer In Reality

Transformers are a common model that is used in modern AI models (Radford et al., 2019; Brown et al., 2020). There are various examples that include the use of transformers. It would not be wrong to say that transformers are the backbone for most of the AI systems, driving applications from conversational agents (ChatGPT, Gemini) to code generation tools (GitHub Copilot) and machine translation. For instance, below is explained how

OpenAI’s ChatGPT uses transformers in text generation.

ChatGPT is built on a Transformer-based model, specifically the decoder part of the original Transformer architecture. The process starts from the tokenisation, where the user inputs data into the model. Each token is mapped to a vector using a learned embedding matrix. Since transformers use parallel processing as discussed earlier, positional encoding is used to capture the order of words in a

sequence. Then the data passes through the decode layers of transformers. By the time the data passes through the decode layers, we are left with hidden vectors, one for each token in the input sequence. The hidden layer is denoted by:

$$H = [h_1, h_2, \dots, h_t] \text{ (shape: } t \times d_{\text{model}})$$

Here, the h_t represents the latest token. After this process, the token is passed through a distribution over the vocabulary. First, a linear transformation is applied using the weight matrix W_{vocab} . This maps the hidden layer from R_{dmodel} to RV , where V is the vocabulary size. Then, the process of converting the defined logits into probabilities. This is carried out by the softmax function:

$$P(\text{next token} = v_i | x_1, \dots, x_t) = \frac{\text{elogs}_i}{\sum_{j=1}^V \text{elogs}_j}$$

Once the model has all the probabilities, it uses a decode strategy to choose the right token, often dependent on parameters like temperature scaling, greedy decoding, top-k sampling, and more. It generates one token at a time in an autoregressive loop. Similar transformer mechanisms underpin other applications, such as Google's Gemini for dialogue and GitHub Copilot for code generation.

Another model that uses transformers is BERT (bidirectional encoder representations from transformers), but in a different way (Devlin et al., 2019). Unlike GPT, which uses a decode model, BERT uses an encoder-only transformer model, primarily for understanding tasks – not generation. The BERT first takes the token by a WordPiece tokenization (a subword-based tokenizer). Then it adds special tokens: [CLS] for starting and [SEP] for ending. After that, each token is converted into a vector representation, which is the sum of the token embedding and the segment embedding. These are passed to the transformer encoder that consists of a self-attention mechanism, feed-forward neural networks, residual connections, and layer normalization. Self-attention, as mentioned earlier, captures each token regardless of its position. In a bidirectional attention, each word is understood depending on the context on both the left and right sides. BERT uses Masked Language Modeling (MLM) for processing data. In this process, the model randomly masks 15% of input tokens during training. Then it tries to predict the original token from the masked content. For example:

Input: He sat on the [MASK].

Output: seat

To understand the sentence-level understanding, the model uses a pretraining task called Next Sentence Prediction (NSP). In this model, the model is given two sentences. A and B, and is trained to predict the sentence B that follows A in the original text. Then comes the stage where fine-tuning takes place. In this phase, the model is adjusted for specific tasks, contributing to the development of a complete end-to-end system.

Other than that, GPT-2 and GPT-3 are also using transformers for text generation. They use transformer decoders, which contain Masked Multi-Head Self-Attention, Feed-Forward Neural Network, Residual Connections, and Layer Normalization. Masked Multi-Head Self-Attention is the core mechanism that allows the model to attend to previous tokens. After that, two individual networks are applied to the tokens, done by a Feed-Forward Neural Network. On the top, Residual Connections and Layer Normalization runs, which ensure stable training and better gradient flow. Once the model has processed the input, it starts to predict one token at a time. The model uses a transformer stack to take the initial prompt and generate a probability distribution over the vocabulary. Then the new token is processed, and the cycles loop around until the task is finished. This process is known as Autoregressive generation.

Lastly, Text-to-Text Transfer Transformer (T5) also makes use of a transformer. Unlike GPT and BERT, T5 uses both an encoder and a decoder. This means that T5 can perform all kinds of tasks, including translation, summarization, or classification problems. For example, in T5, instead of building a separate classifier for sentiment analysis, the input is as follows:

“Classify sentiment: The movie was amazing” → “positive”.

The training objective of T5 is a variant of denoising autoencoding, where parts of the input text are masked and the model must reconstruct the missing ones, as discussed earlier.

Challenges

Transformers with benefits also have drawbacks. One of which is the Quadratic Complexity of Self-

Attention. The model uses $O(n^2)$ time and memory complexity, which makes it computationally expensive and money-intensive. Moreover, long-range dependencies are harder without Longformer, Reformer, and FlashAttention. Furthermore, transformers also suffer from training challenges. As models like GPT use autoregressive approaches, which don't necessarily require deep comprehension or understanding, may lead to surface-level fluency without understanding the deeper understanding of tokens. Transformers use large corpora from the internet, which can be biased or toxic. This can result in amplifying harmful stereotypes and misinformation.

In addition to that, transformers have a limited context window, which prevents them from accessing earlier parts of long conversations. Often with loss of coherence, drift off-topic, or introduce hallucinated facts, transformers can generate low-quality output and require a high depth of prompt. Transformers can also be inconsistent with their outputs. It is difficult to manoeuvre the outputs in a precise way. Cost is another problem related to transformers. As of model T5-11B with 11 billion parameters, it utilizes \$1-2 million+ to run on the cloud. One of the major problems is ethics. As transformers learn from the publicly available text, it can contain misinformation, plagiarism, racism, political bias, and much more.

Future Direction

The future of transformers is very promising. Till now, transformers require a large amount of capital to run, but research is underway to invent efficient variants such as Longformers, Reformers, and Performers to reduce memory and compute requirements. Besides this, instead of relying on redefined parameters, transformers are aligned with external knowledge sources. For example, Retrieval-Augmented Generation (RAG) is implemented, which integrates search and retrieval into the generation pipeline. Research is in progress to find effective ways to reduce bias and hallucinations in text generation. Techniques like reinforcement learning with human feedback (RLHF), low-rank adaptation (LoRA), and prompt-tuning help improve controllability.

Conclusion

In the end, it is to be concluded that transformers have changed the way text is generated, replacing

older models with self-attention. It can capture long-term dependencies, scale efficiently, and generalise tasks that create state-of-the-art models like ChatGPT and BERT. Despite the challenges, research into efficient architectures and responsible AI practice continues to advance this field. With modifications, transformers would be viewed as the backbone for future NLP and beyond (Vaswani et al., 2017).

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems, 30. <https://arxiv.org/abs/1706.03762>
2. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of deep bidirectional transformers for language understanding*. Proceedings of NAACL-HLT, 4171–4186. <https://arxiv.org/abs/1810.04805>
3. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners*. OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
4. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). *Language models are few-shot learners*. Advances in Neural Information Processing Systems, 33, 1877–1901. <https://arxiv.org/abs/2005.14165>
5. Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. Neural Computation, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
6. Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). *A neural probabilistic language model*. Journal of Machine Learning Research, 3, 1137–1155. <http://www.jmlr.org/papers/v3/bengio03a.html>